System Documentation

Digiflash – PETC Data Submission Client

DOTr IT Provider Accreditation – Deliverable #3

1. Document Control

Field	Value
Document title	System Documentation – Digiflash PETC Data Submission Client
Document version	0.2 (Draft)
Document date	2026-06-03
Product name	Digiflash
IT Provider	Digiflash
Prepared by	Christian Deiniel Y. Silerio (Lead Developer, Digiflash)
Prepared for	Department of Transportation (DOTr) / Land Transportation Office (LTO)
Client Application version	`0.1.0` (from desktop/package.json)
Source commit	resolved by `git rev-list -n 1 accreditation-2026-06-03`

1.1 Revision History

Version	Date	Author	Summary
0.1	2026-06-01	C. Silerio	Initial draft for DOTr accreditation submission.
0.2	2026-06-03	C. Silerio	Re-aligned with the implemented cloud-mediated LTMS path: cloud is the single LTMS / IRDS submitter; desktop talks only to the cloud and S3 (presigned PUT). Updated brand to **Digiflash**, install paths to `Digiflash`. Corrected `cloud/backend/` references to the active `cloud/src/`. Updated migration list (`V1__init` , `V2__submissions` , `V3__submission_cec_fields`). Added `submissions/reconciler.py` and `cloud_client.py` to the sidecar sub-program list. Tightened the update-distribution and code-signing sections to match the present, deferred posture. Source commit replaced with the submission tag name.

1.2 Scope of this Document

This document satisfies the System Documentation requirement of the DOTr IT Provider accreditation, which calls for:

1. A complete description of the executable file of the Client Program and its System Security Policy.
2. A declaration and list of the main application sub-programs and other files associated with the submitted Client Application.
3. Screenshots of folder location, file location, and size for each system file.

The "Client Application" referred to throughout this document is the **Digiflash desktop application** installed at each accredited Private Emission Testing Center. The Digiflash cloud service is the entity that holds LTMS / IRDS credentials and submits to LTMS / IRDS on the desktop's behalf; it is included in this document where the desktop intersects with it. Both pieces of software are produced and operated by Digiflash.

---

2. System Overview

2.1 Product Description

Digiflash is a multi-tenant emission-testing platform for accredited Private Emission Testing Centers in the Philippines. At each center, a Windows desktop application captures emission readings from supported analyzer hardware, photographs the vehicle through a USB webcam, merges the captured data with vehicle and owner data retrieved from LTMS / IRDS **through the Digiflash cloud**, and submits the completed emission test record to LTMS / IRDS **also through the Digiflash cloud**. A printed Certificate of Emission Compliance (CEC) is issued to the vehicle owner once LTMS returns the certificate key, OR number, DERMALOG token, and 60-day validity window.

The desktop application is the source of truth for test records at each center. The cloud service holds the single whitelisted public IP that LTMS and IRDS allow inbound from; no individual PETC IP is whitelisted by either registry.

2.2 Three-Tier Architecture

```
+--------------------------------------------------------------+
| Tier 1 — Hardware at the Testing Bay                         |
|   * Emission analyzer (gas or diesel, serial RS-232/USB)     |
|   * Webcam(s) for vehicle and bay photos                     |
|   * Receipt / CEC printer                                    |
+--------------------------------------------------------------+
                              |
                              v  (USB / Serial COM)
+--------------------------------------------------------------+
| Tier 2 — Client Application (Desktop, at each PETC)          |
|   * Electron main process     (Node.js / TypeScript)         |
|   * React renderer            (TypeScript / React 18)        |
|   * Python FastAPI sidecar    (Python 3.11)                  |
|   * Local SQLite database     (source of truth)              |
```

```
|   Loopback only: 127.0.0.1:8765                                |
+----------------------------------------------------------------+
                              |
                              v  (HTTPS, outbound only)
+----------------------------------------------------------------+
| Tier 3 — Digiflash Cloud (Spring Boot + Postgres + S3, AWS)    |
|   * /api/photos/presign   short-lived S3 PUT URL               |
|   * /api/submissions      enqueue + poll LTMS submission       |
|   * /api/registry/...     proxy LTMS / IRDS lookups            |
|   * NAT Gateway elastic IP — the single IP whitelisted by      |
|     LTMS and IRDS                                              |
+----------------------------------------------------------------+
                              |
                              v  (HTTPS, from the cloud only)
+----------------------------------------------------------------+
| Tier 4 — Government Registries                                 |
|   * LTMS  (LTO)                                                |
|   * IRDS  (Stradcom)                                           |
+----------------------------------------------------------------+
```

2.3 Source-of-Truth Statement

The local SQLite database (`petc.db`) on each Digiflash desktop is the authoritative record of emission tests conducted at that center. The center can continue capturing tests during a temporary cloud or LTMS outage; queued tests are pushed to the cloud when connectivity is restored, and the cloud completes the LTMS submission asynchronously. The CEC is issued only after LTMS returns `ACCEPTED` ; if the 60-second short-poll times out the submission enters `WAITING_FOR_LTMS` and a background reconciler on the desktop resolves it when the cloud reports a terminal state.

---

3. Executable File Description – Client Program

The Client Application delivered at each PETC consists of three packaged components, all bundled inside one Windows installer produced by `electron-builder` .

3.1 Primary Executable

Property	Value
Filename	`Digiflash.exe`
Type	Electron desktop application
Runtime	Embedded Chromium + Node.js (Electron 28+)
Source entry point	desktop/electron/main.ts

Preload script	desktop/electron/preload.ts
Build configuration	desktop/package.json, desktop/installer/
Default install location	`C:\Program Files\Digiflash\`

The Electron main process is responsible for: creating the main application window, spawning and supervising the Python sidecar subprocess, mediating IPC between renderer and main, and applying signed auto-updates pulled from the operator cloud update channel.

3.2 Embedded Sidecar Executable

Property	Value
Filename	`petc-sidecar.exe` (PyInstaller-frozen)
Type	Local HTTP service
Runtime	Embedded Python 3.11
Source entry point	desktop/sidecar/petc/service.py
Bind address	`127.0.0.1:8765` (loopback only — no LAN exposure)
Lifecycle	Spawned by Electron main at app start; terminated on app exit
Build configuration	desktop/pyproject.toml, desktop/installer/

The sidecar exposes a private FastAPI HTTP server that the React renderer calls for: authentication, vehicle lookup against the LTMS/Stradcom registry, analyzer reading capture, camera and printer control, CEC generation, and LTMS submission.

3.3 Bundled User Interface

Property	Value
Type	Static React + Vite build (HTML, JS, CSS)
Runtime	Loaded into the Electron renderer process
Source entry point	desktop/renderer/src/main.tsx
Application shell	desktop/renderer/src/App.tsx
Built asset location	`resources/app.asar/renderer/dist/` (inside installer)

3.4 Runtime Data Locations

Asset	Location
Local SQLite database	`%APPDATA%\Digiflash\petc.db`
Photo storage	`%APPDATA%\Digiflash\photos\`

CEC PDFs (rendered locally after LTMS accepts)	`%APPDATA%\Digiflash\cec\`
Application logs	`%APPDATA%\Digiflash\logs\`
User-specific settings	`%APPDATA%\Digiflash\settings\`

The `%APPDATA%\Digiflash\` directory is created by the installer with NTFS ACLs restricting access to the operating-system user account that runs the application.

---

4. System Security Policy

This section is the formal Security Policy of the Client Application required by the DOTr accreditation. Each control area below describes the policy and the source-code location where it is enforced.

4.1 Authentication

- Operators log in to the Client Application via the local sidecar endpoint `POST /auth/login`.
- User passwords are stored as bcrypt hashes via `passlib` and are never persisted in plaintext.
- A successful login issues a short-lived JWT session token, held in the renderer's Zustand state for the duration of the session.
- Cloud mirror requests carry an `X-Center-Key` header whose hash is verified server-side against the `licenses.key_hash` column on the operator cloud.
- Enforced in: desktop/sidecar/petc/api/, desktop/sidecar/petc/service.py.

4.2 Authorization (Role-Based Access Control)

- Three roles are recognised by the Client Application:
    - **Encoder** – performs tests, captures readings and photos, submits to LTMS.
    - **Supervisor / Admin** – reviews tests, manages operators, configures hardware.
    - **Platform Super Admin** – operates only on the cloud portal; no rights inside the Client Application.
- Authorization decisions are enforced at every protected sidecar route.
- Cloud-side enforcement is performed by the stateless JWT filter in the Spring Security configuration: cloud/src/main/java/com/petc/auth/SecurityConfig.java and the per-request `X-Center-Key` filter in cloud/src/main/java/com/petc/ingest/CenterKeyValidator.java.

4.3 Tenant Isolation

- Each accredited PETC corresponds to exactly one tenant on the operator cloud.
- Every cloud request originating from a Client Application is bound to a tenant via the `X-Center-Key` header. The validator resolves the key to a `tenantId`, and every

downstream query is scoped by that `tenantId`.
- A tenant's records are never visible to operators of another tenant.

4.4 Data Protection

- The local SQLite database is stored under the per-user `%APPDATA%\PETC\` directory, inheriting NTFS access controls of that user account.
- The cloud-side API key (`PETC_CLOUD_KEY`) is stored in the operating system credential store / encrypted configuration file, never in plaintext source.
- Test photos are written to the local photos directory and are referenced by their SHA-256 content hash.
- All outbound communication to LTMS, Stradcom, and the PETC operator cloud is performed exclusively over HTTPS in production deployments.

4.5 Audit Logging

- Every state-changing action in the Client Application — operator login, test creation, photo capture, LTMS submission attempt, CEC issuance — is written to a local audit trail (the `app_settings` and outbox tables and a rolling log file).
- LTMS submission attempts log: timestamp, operator user ID, plate number, attempt outcome, and any error returned by the registry.
- When the cloud mirror is enabled, audit events are forwarded to the cloud `audit_logs` table.

4.6 Network Posture

- The Python sidecar binds to `127.0.0.1` only; it is not reachable from the local network or the internet.
- Outbound network connections from the Client Application are restricted to:
  - The Digiflash cloud API (`api.digiflash.ph` — illustrative; the production hostname is fixed at deployment time) for presign, submission, polling, and registry-lookup proxy calls.
  - The S3 bucket hostname (`*.s3.<region>.amazonaws.com`) for direct presigned PUT of photo bytes.
  - The application update feed (see §4.7).
  - Standard OS endpoints (Windows Update, Windows Time / NTP).
- The Client Application does **not** open any direct connection to LTMS or Stradcom. Those endpoints are reached only from the Digiflash cloud's NAT gateway, whose elastic IP is the single source IP whitelisted by LTMS / IRDS.
- No inbound listener is exposed by the Client Application beyond the loopback sidecar.

4.7 Update Distribution

- Application updates are distributed via an `electron-updater` generic feed configured at build time in desktop/installer/electron-builder.yml. At the time of this submission the feed

URL is a placeholder (`https://releases.petc.example.com`); the production feed URL is fixed at deployment time.

- Code-signing of the Windows installer and signature verification of update manifests are scheduled work items. They are **not** in place at the time of this submission and are tracked as a release-blocking prerequisite before commercial rollout. The accreditation review version of the installer is unsigned and installs only with explicit operator confirmation.

---

5. Declaration and List of Main Sub-Programs

5.1 Declaration

I, **Christian Deiniel Y. Silerio**, in my capacity as Lead Developer of the PETC Data Submission SaaS, declare that the sub-programs and files enumerated in this Section 5 and in Section 6 constitute the complete set of sub-programs and associated files that comprise the Client Application submitted under this accreditation. No undeclared executable code is included in the installer beyond standard third-party runtime dependencies declared in desktop/package.json and desktop/pyproject.toml.

Signed: ______________________ Date: _______________

5.2 Electron Main Process (Node.js 20 / TypeScript)

Source root: desktop/electron/

File	Purpose
`main.ts`	Application bootstrap; creates the browser window, spawns the Python sidecar, applies updates.
`preload.ts`	Secure IPC bridge between renderer and main process.
`bridge.d.ts`	TypeScript declarations for the renderer-side IPC contract.
`tsconfig.json`	TypeScript compiler configuration for the main process.

5.3 React Renderer (TypeScript / React 18)

Source root: desktop/renderer/src/

Sub-program	Purpose
`pages/auth/`	Operator login screen.
`pages/test/`	"Run Test" workflow: plate lookup, fuel-type selection, analyzer readings capture.
`pages/upload/`	Six-step LTMS upload wizard: vehicle, owner, engine flags + readings, technician, photos, review & submit.
`pages/history/`	Past tests, filtering, re-printing of CEC.

`pages/settings/`	Analyzer / camera / printer configuration; COM port discovery.
`pages/analytics/`	Local center analytics dashboard.
`store/`	Zustand state (auth, current test, settings).
`api/`	Typed HTTP client for the local sidecar.

5.4 Python Sidecar (Python 3.11 / FastAPI)

Source root: desktop/sidecar/petc/

Sub-program	Purpose
`service.py`	Sidecar entry point — wires analyzer, camera, printer, gov adapter, cloud-sync pusher, and submission reconciler; starts the FastAPI server on loopback.
`api/server.py`	All sidecar HTTP routes (auth, vehicle / driver lookup, tests, photo capture, upload submission, CEC preview + print, ports, settings).
`cloud_client.py`	HTTP client for the Digiflash cloud (`/api/photos/presign` , `/api/submissions` , `/api/submissions/{id}` , `/api/registry/vehicle/{plate}` , `/api/registry/driver/{lic}`).
`submissions/reconciler.py`	Daemon thread that polls the cloud every 30 s for any local `LtmsSubmission` row in `PENDING` or `WAITING_FOR_LTMS` , updates it on a terminal cloud state, and renders the CEC PDF on `ACCEPTED` .
`analyzer/`	Serial hardware adapters (see 5.5).
`camera/`	Webcam capture (OpenCV) + mock.
`printer/`	Thermal receipt printer integration + mock.
`gov/`	Local LTMS / IRDS adapter (mock for offline / dev path; Stradcom stub). In the production cloud-mediated path these are reached via the cloud rather than the local sidecar.
`cec/pdf.py`	Two-halves-per-A4 CEC PDF renderer (customer copy + center copy on a single sheet).
`db/models.py` , `db/session.py` , `db/migrations/`	SQLAlchemy ORM models, session factory, Alembic env.
`cloud_sync/pusher.py`	Outbound mirror pusher for local-to-cloud sync of completed tests + photos.
`queue/outbox.py`	Outbox pattern with retry + dead-letter.
`sync/`	Reserved for future helpers (empty stub package at the time of submission).

5.5 Hardware Analyzer Adapters (Python)

Source root: desktop/sidecar/petc/analyzer/

Adapter file	Hardware	Protocol
`mock.py`	Mock analyzer (test fixture; used when `PETC_ANALYZER=mock`)	In-memory simulated readings
`base.py`	Analyzer abstract base classes	—
`serial_base.py`	Serial-COM abstraction	RS-232 / USB-serial
`builder.py`	Adapter factory; chooses adapter by `PETC_ANALYZER` value	—
`ascii_gas.py`	Generic gas analyzer	ASCII delimited, CRLF terminated, passive push
`binary_diesel.py`	Generic diesel opacimeter	Binary framed, CRC-16/MODBUS, polled with trigger byte `0x05`
`fty_opacimeter.py`	FTY-100 diesel opacimeter	Diesel opacity, polled
`fofen_gas.py`	Fofen petrol gas analyzer	Fofen binary framing
`fofen_ascii.py`	Fofen ASCII variant	ASCII variant
`fofen_framing.py`	Fofen frame parser	Binary frame decode

Frame formats and unit-test fixtures for the ASCII gas and binary diesel adapters are documented in the project README and reproduced in Section 6.4.

---

6. Associated Files

6.1 Configuration Files

File	Purpose
desktop/package.json	Electron / renderer Node dependencies and build scripts.
desktop/pyproject.toml	Python sidecar package metadata and dependencies.
desktop/installer/	electron-builder and PyInstaller specifications.
cloud/src/main/resources/application.yml	Spring Boot cloud configuration (datasource, JWT, S3, gov adapter, submission retry policy).
docker-compose.yml	Local development services (Postgres, MinIO, Redis).

6.1.1 Environment Variables

Variable	Default	Purpose
`PETC_DATA_DIR`	`%APPDATA%\Digiflash`	Root directory for SQLite, photos, CEC PDFs, logs.
`PETC_PORT`	`8765`	Sidecar HTTP port (loopback only).
`PETC_ANALYZER`	`mock`	Adapter type: `mock`, `serial_gas`, `serial_diesel`, `fty_opacimeter`, `fofen_gas`, `fofen_ascii`.
`PETC_ANALYZER_PORT`	`COM1`	COM port name (e.g. `COM3`, `/dev/ttyUSB0`).
`PETC_ANALYZER_BAUD`	`9600`	Baud rate (use 19200 for diesel).
`PETC_GOV_MOCK`	`true` (dev)	Use the **local** mock gov client (offline / dev path). In production this is false and gov calls go via the cloud.
`PETC_CLOUD_URL`	unset (prod: set)	Digiflash cloud base URL. When set, the desktop submits to LTMS / IRDS via the cloud and proxies registry lookups through the cloud. When unset, the desktop falls back to the local mock-gov path.
`PETC_CENTER_ID`	unset	Center identifier issued at commissioning.
`PETC_CLOUD_KEY`	unset	Per-center API key carried on every cloud request in the `X-Center-Key` header.

6.2 Database Files

6.2.1 Local (SQLite, source of truth)

- Database file: `%APPDATA%\Digiflash\petc.db`
- Schema migrations: desktop/sidecar/petc/db/ (Alembic).
- Core tables: `users`, `emission_tests`, `gas_test_results`, `diesel_test_results`, `test_photos` (incl. `s3_key`, `uploaded_at`), `ltms_submissions` (incl. `cloud_submission_id`, `or_no`, `dermalog_token`, `valid_from`, `valid_until`, `pdf_path`), `vehicles_cache`, `drivers_cache`, `receipts`, `gov_outbox`, `cloud_outbox`, `app_settings`, `audit_log`.

6.2.2 Cloud (PostgreSQL)

Schema migrations under cloud/src/main/resources/db/migration/:

Migration	Purpose
`V1__init.sql`	Core tables: `tenants`, `users`, `refresh_tokens`, `audit_log`, mirror tables (`mirror_emission_tests`, `mirror_test_photos`, `mirror_ltms_submissions`). Row-Level Security policies.

`V2__submissions.sql`	`center_licenses` (bcrypt-hashed X-Center-Key per tenant) + `submissions` (LTMS submission queue with state machine, attempts, backoff).
`V3__submission_cec_fields.sql`	Adds the CEC presentation fields LTMS returns: `or_no`, `dermalog_token`, `valid_from`, `valid_until`.

6.3 Runtime Data

Asset	Location	Notes
Photo storage	`%APPDATA%\Digiflash\photos\`	Files named `<photoId>.jpg` under the per-test folder. Also uploaded to S3 by presigned PUT once the wizard reaches step 6.
CEC PDFs	`%APPDATA%\Digiflash\cec\`	Two-halves-per-A4 PDF per accepted submission; filename is the submission UUID.
Application logs	`%APPDATA%\Digiflash\logs\`	Rolling daily files; retained 30 days.

6.4 Test Fixtures (for inspector verification)

Fixture	Purpose
desktop/tests/fixtures/gas_pass.txt	ASCII gas frame with all fields.
desktop/tests/fixtures/gas_no_optional.txt	ASCII gas frame, required fields only.
desktop/tests/fixtures/diesel_pass.bin	Binary diesel frame with valid CRC.

Inspectors can verify analyzer parsing offline by running `pytest desktop/tests -q` against these fixtures; no physical analyzer is required.

---

7. Folder and File Inventory (Screenshots)

DOTr requires "Screenshots of folder location, file(s) location and size for each and every system files." This section lists every path that must be captured. Screenshots are produced separately (see follow-up task) and inserted under each numbered heading below.

7.1 Capture Procedure

For each item in the checklist below:

1. Open the path in **Windows File Explorer**.
2. Switch to **Details** view (View ribbon → Details).
3. Ensure these columns are visible: **Name**, **Date modified**, **Type**, **Size**.
4. Capture a full-window screenshot (Win+Shift+S or Alt+PrtScn).

5. Insert the image beneath the corresponding heading in this document.

Where command-line listings are preferred, run in PowerShell:

```
Get-ChildItem -Force -Recurse <path> |
  Select-Object FullName, Length, LastWriteTime |
  Format-Table -AutoSize
```

and paste the formatted output.

7.2 Screenshot Checklist

#	Path	Captured
1	`C:\Program Files\Digiflash\` (root install directory)	☐
2	`C:\Program Files\Digiflash\resources\app.asar.unpacked\sidecar\` (sidecar exe folder)	☐
3	`C:\Program Files\Digiflash\resources\app.asar\renderer\dist\` (renderer assets)	☐
4	`%APPDATA%\Digiflash\` (runtime data root)	☐
5	`%APPDATA%\Digiflash\petc.db` (SQLite database, showing file size)	☐
6	`%APPDATA%\Digiflash\photos\` (photo storage)	☐
7	`%APPDATA%\Digiflash\cec\` (CEC PDFs)	☐
8	`%APPDATA%\Digiflash\logs\` (application logs)	☐
9	Source: `desktop\electron\` (main process source)	☐
10	Source: `desktop\renderer\src\pages\` (UI sub-programs)	☐
11	Source: `desktop\sidecar\petc\` (sidecar root)	☐
12	Source: `desktop\sidecar\petc\analyzer\` (all analyzer adapter files)	☐
13	Source: `desktop\sidecar\petc\submissions\` (reconciler)	☐
14	Source: `desktop\sidecar\petc\cec\` (CEC generation)	☐
15	Source: `desktop\sidecar\petc\db\` (DB models and migrations)	☐
16	Source: `cloud\src\main\java\com\petc\submissions\` (cloud LTMS submitter)	☐
17	Source: `cloud\src\main\java\com\petc\gov\` (cloud LTMS / IRDS adapters)	☐
18	Config: `desktop\package.json`, `desktop\pyproject.toml`	☐
19	Build: `desktop\installer\`	☐

20	Cloud config: `cloud\src\main\resources\application.yml`	☐
21	Cloud migrations: `cloud\src\main\resources\db\migration\`	☐

7.3 Screenshot Placeholders

> *The numbered headings below correspond to each row in the checklist. Insert the captured screenshot under the matching heading.*

7.3.1 Install root – `C:\Program Files\Digiflash\`

(screenshot to be inserted)

7.3.2 Sidecar executable folder

(screenshot to be inserted)

7.3.3 Renderer asset folder

(screenshot to be inserted)

7.3.4 Runtime data root – `%APPDATA%\Digiflash\`

(screenshot to be inserted)

7.3.5 Local SQLite database – `petc.db`

(screenshot to be inserted)

7.3.6 Photo storage directory

(screenshot to be inserted)

7.3.7 CEC PDF storage directory

(screenshot to be inserted)

7.3.8 Application log directory

(screenshot to be inserted)

7.3.9 Electron main-process source

(screenshot to be inserted)

7.3.10 React renderer pages

(screenshot to be inserted)

7.3.11 Python sidecar root

(screenshot to be inserted)

7.3.12 Hardware analyzer adapters

(screenshot to be inserted)

7.3.13 Submission reconciler

(screenshot to be inserted)

7.3.14 CEC generation folder

(screenshot to be inserted)

7.3.15 Database models and migrations folder

(screenshot to be inserted)

7.3.16 Cloud LTMS submitter (`cloud/src/.../submissions/`)

(screenshot to be inserted)

7.3.17 Cloud LTMS / IRDS adapters (`cloud/src/.../gov/`)

(screenshot to be inserted)

7.3.18 Build configuration files

(screenshot to be inserted)

7.3.19 Installer specification folder

(screenshot to be inserted)

7.3.20 Cloud `application.yml`

(screenshot to be inserted)

7.3.21 Cloud Flyway migration folder

(screenshot to be inserted)

---

8. Version and Build Information

Item	Value
Client Application version	`0.1.0` (from [desktop/package.json])
Source repository submission tag	`accreditation-2026-06-03` (annotated; commit resolved by `git rev-list -n 1 accreditation-2026-06-03`)

Node.js (renderer / main process build)	20.x LTS
Python (sidecar runtime)	3.11
Java (cloud build)	21
Build tooling	`npm`, `electron-builder`, `PyInstaller`, Gradle 8.10
Target operating system	Windows 10 / 11 (64-bit)
Installer format	NSIS (electron-builder default)

---

9. References

- Project Proposal: *Emission Testing Center Data Submission SaaS – Project Proposal*, Feb 26, 2026.
- Repository README.md: desktop setup, analyzer protocols, mock LTMS workflow, cloud mirror testing.
- desktop/electron/main.ts – Electron entry point.
- desktop/sidecar/petc/service.py – Python sidecar entry point.
- desktop/sidecar/petc/api/server.py – sidecar HTTP routes.
- desktop/sidecar/petc/cloud_client.py – Digiflash cloud client.
- desktop/sidecar/petc/submissions/reconciler.py – background reconciler for `WAITING_FOR_LTMS`.
- desktop/sidecar/petc/cec/pdf.py – CEC PDF renderer.
- desktop/renderer/src/main.tsx – React renderer entry point.
- cloud/src/main/java/com/petc/PetcApplication.java – Spring Boot cloud entry point.
- cloud/src/main/java/com/petc/auth/SecurityConfig.java – cloud security configuration.
- cloud/src/main/java/com/petc/submissions/SubmissionsController.java – cloud LTMS submission endpoint.
- cloud/src/main/java/com/petc/gov/GovRegistryClient.java – LTMS / IRDS adapter interface.
- Companion accreditation documents: `01-client-application-manual.md`, `02-setup-and-network-layout.md`, `04-source-code/`, `05-cec-samples/`, `06-network-architecture.md`.

End of System Documentation – PETC Data Submission SaaS Client Application.